Lec-16: Operator Overloading in C++

What is `std::map`?

`std::map<K,V>` is an associative container that stores key–value pairs in **sorted order by key**:

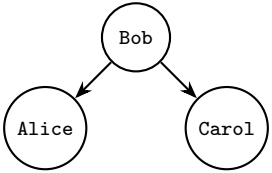
```
std::map<std::string, int> ages;
ages["Alice"] = 20;
ages["Bob"] = 22;
ages["Carol"] = 21;

// iteration visits keys in sorted order:
// Alice -> Bob -> Carol
for (int i = 0; i < ages.size(); i++) {
    // ...
}
```

Internally, `std::map` organises its entries in a **balanced binary search tree** (typically a red-black tree).

This is what gives `std::map` $O(\log n)$ lookup, insertion, and deletion — much better than scanning every element.

How does a `std::map` stay sorted?

A binary search tree maintains an ordering invariant: for every node, all keys in its left subtree are *smaller* and all keys in its right subtree are *larger*.



To place a new key, the tree compares it against existing nodes using the < operator at each step:

- Is the new key *less than* this node? Go left.
- Otherwise? Go right.

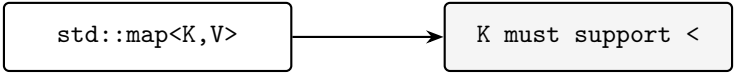
For `std::string`, the < operator is already defined. But what about *our own* types?

The problem: custom keys in a `std::map`

What if we want to use a `Student` as a map key?

```
std::map<Student, double> grades;
Student alice("Alice", 101);
grades[alice] = 3.8; // compiler ERROR!
```

The compiler rejects this because `std::map` needs to compare `Student` objects using `operator<`, and we have not defined one.



This is the first clue that user-defined types sometimes *must* behave like built-in types — and that is exactly what **operator overloading** lets us do.

Functions vs. operators – what's the difference?

Consider a `Money` type. Two ways to add:

```
// function-based
Money result = add(a, b);
```

Feels like an arbitrary function call. No hint that `Money` has numeric-like behaviour.

```
// operator-based
Money result = a + b;
```

Immediately communicates that `Money` values can be combined the way numbers can.

"Operators allow you to convey meaning about types that functions don't."

A concrete need: `std::min` for a custom type

Suppose we want the minimum of two `Student` objects:

```
Student alice;
Student bob;
auto m = std::min<Student>(alice, bob);
```

`std::min` internally does `a < b ? a : b`.

If `Student` has no `operator<`, the compiler rejects the call.

Takeaway: Many generic algorithms and containers *require* that our types support specific operators.

What are operators?

Operators are symbols that perform operations on values, objects, or types and produce a new value or effect.

Values	Objects	Types
`3 + 4`	`a < b`	`sizeof(T)`

Overloading lets us define what an operator means for *our own* types — the compiler calls our implementation whenever that operator is used.

Which operators *cannot* be overloaded?

A small set of operators is off-limits:

Operator	**Symbol**	**Reason**
Scope resolution	`::`	Language structure
Ternary conditional	`?:`	Control flow
Member access	`.`	Language structure
Pointer-to-member	`.*`	Language structure
`sizeof`	`sizeof()`	Compile-time query
`typeid`	`typeid()`	RTTI query
Cast operators	`static_cast` etc.	Type system

Everything else (arithmetic, relational, bitwise, stream I/O, subscript, function-call, . . .) *can* be overloaded.

Syntax at a glance

The general form:

```
return_type operator<symbol>(parameter_list);
```

- `<symbol>` is the operator token (<, +, ==, «, . . .).
- The parameter list depends on whether the overload is a **member** or a **non-member** function.

Binary and unary operators

Operators come in two flavours:

Binary — two operands:

```
a + b // arithmetic
a < b // relational
a << b // stream / shift
```

As a **member**: one parameter (LHS is `*this`).
As a **non-member**: two parameters.

Unary — one operand:

```
-a // prefix negate
!a // prefix logical NOT
++a // prefix increment
a++ // postfix increment
```

As a **member**: zero parameters.
As a **non-member**: one parameter.

You can only overload operators that already exist in the grammar — you cannot invent new operator tokens (e.g. no ** for exponentiation).

Prefix vs. postfix: the dummy `int`

How does the compiler tell `++a` apart from `a++`? C++ uses a **dummy `int` parameter** to mark the postfix form:

```
class Counter {
    int n;
public:
    Counter(int n) : n(n) {}

    Counter& operator++() // prefix ++c
    { ++n; return *this; }

    Counter operator++(int) // postfix c++
    { Counter old = *this; ++n; return old; }
};
```

Key differences:

- Prefix returns a *reference* (the modified object).
- Postfix returns *by value* (a copy of the old state).
- The `int` argument is never used — it only disambiguates the signature.

Rules

C++ imposes several constraints on operator overloading:

1. At least one operand must be a user-defined type.

You cannot redefine what `int + int` means. This ensures that C++ is *extensible but not mutable* (Stroustrup).

2. You cannot create new operator tokens.

Only the operators listed in the grammar can be overloaded. If you need exponentiation, write a named function: `pow(a,b)`.

3. Predefined meanings do not transfer automatically.

Defining `operator+` and `operator=` does *not* give you `operator+=`. Each operator must be defined separately.

4. You cannot change arity or precedence.

A binary operator stays binary; `*` cannot become ternary. The expression `a+b*c` always parses as `a+(b*c)`.

Some operators *must* be members

C++ requires four operators to be **non-static member functions**:

Operator	Symbol	Reason
Assignment	=	Must modify `*this`
Subscript	[]	Provides lvalue access to elements
Function call	()	Makes objects callable (functors)
Member access	->	Smart-pointer dereferencing

All four share a common trait: they fundamentally operate on the *identity* of the left-hand object, not just its value. Making them non-member would break that semantic connection.

All other operators may be either member or non-member — and for symmetric binary operators we prefer non-member.

Deleting operators

Three operators have **predefined meanings** for class objects even if you never define them: `=` (copy assignment), `&` (address-of), and `,` (sequencing).

If those defaults are inappropriate, you can *delete* them:

```
class NoCopy {
public:
    NoCopy() = default;

    // forbid copying
    NoCopy(const NoCopy&) = delete;
    NoCopy& operator=(const NoCopy&) = delete;
};
```

```
NoCopy a;
NoCopy b = a; // ERROR: copy constructor deleted
a = b; // ERROR: operator= deleted
```

This is the modern C++ way to say "this type cannot be copied" — clear, explicit, and caught at compile time.

The Student class

```
class Student {
private:
    std::string name;
    int rollNumber;
public:
    Student(std::string name, int rollNumber);

    std::string getName() const;
    int getRollNumber() const;

    bool operator<(const Student& rhs) const;
};
```

The `operator<` is declared *inside* the class — this is **member overloading**.

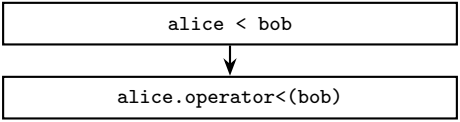
Implementing member operator<

```
bool Student::operator<(
        const Student& rhs) const
{
    return rollNumber < rhs.rollNumber;
}
```

Key observations:

- The left-hand operand is the implicit `*this`.
- The right-hand operand is the explicit parameter `rhs`.
- The trailing `const` promises we do not modify the left-hand object.
- We can access `rhs.rollNumber` directly because both objects share the same class (private access within the class).

How the compiler sees it



The expression `alice < bob` is syntactic sugar for `alice.operator<(bob)`. The compiler rewrites it for us.